Variational Inference for De Finetti Logic

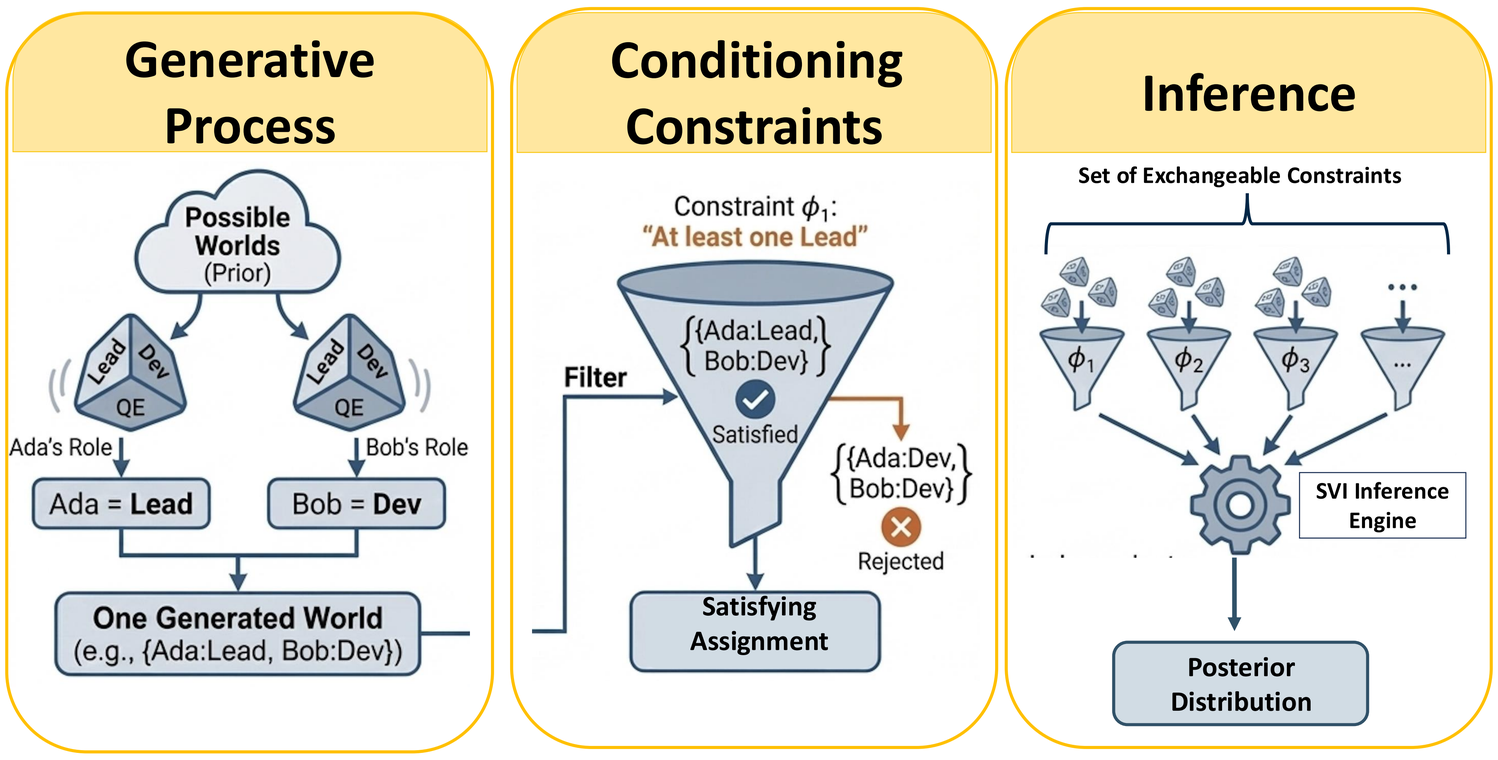
Sami Hadouaj, Ouael Ben Amara, Niccolò Meneghetti

DEARBORN CECS

Motivation & Context

- Traditional databases excel at **storing data** and answering **deterministic queries.**
- We envision a next-generation system: transforming the database from a passive store into an **active probabilistic reasoner** that integrates **inference directly into the query pipeline**.
- **Relational Probabilistic Programming (RPP)** provides the foundation, merging **logic** for domain structure with **probability** for uncertainty.
- To make this practical for real-world data, we build our RPP system **natively inside a database execution engine** and power it with **Stochastic Variational Inference (SVI)**: a **scalable, optimization-based** algorithm for fast, approximate Bayesian learning.

Relational Probabilistic Programming



Main Concepts

Probabilistic Program

A probabilistic program is composed of a ***stochastic generative process*** and a set **of *conditioning observations***.

Pólya urn: evidence reinforces itself

Ada example: red = distracted, green = vigilant

distracted vigilant

1 Start with prior counts

$\alpha_d = 2, \quad \alpha_v = 2$

Initial counts encode prior belief about Ada's vigilance state.

2 Draw one outcome

Example draw: vigilant

The draw is one observation from Ada's current um composition.

3 Return it + add same color

+1

$P_A(v): \quad \frac{2}{4} \rightarrow \frac{3}{5}$

The observed outcome becomes more likely in future draws.

4 Exchangeability: only counts matter

A: B:

$P_A(v, d, v) = P_A(v, v, d)$

Both histories have the same counts: 2 vigilant + 1 distracted.

Order changes; counts stay.

Pólya-Eggenberger urn rule

$$P(\text{next} = r) = \frac{\alpha_r + R_t}{\alpha_r + R_t + \alpha_g + G_t}$$

Here: red = distracted, green = vigilant; R_t and G_t are observed counts.

Connection to De Finetti Logic

$\theta \sim \text{Dir}(\alpha), \quad X[\phi_m] \sim \text{Cat}(\theta)$

The um becomes a Pólya die; DFL adds logical constraints over outcomes drawn from parallel ums/dice.

Takeaway: the urn remembers evidence through counts, not order.

Probabilistic Datalog

We use a restricted version of Probabilistic programming with Datalog[1] to represent probabilistic programs. This is done by defining two main rules that describe our Bayesian pipeline.

Rule1: Defines the instantiation of a die given a domain and a set of hyperparameters

$$\text{lp}(\underline{VarId}, D, p \in \mathcal{S}_{|D|} \sim \mathcal{D}ir[\![H]\!]) \leftarrow \text{dt}(VarId, D, H)$$

Rule2: characterizes the operation of throwing a die and denoting the observation. This is the sampling process

$$\text{obs}(\underline{VarId, ObsId}, v \in D \sim \mathcal{C}at[\![P]\!]) \leftarrow \text{lp}(VarId, D, P), \text{sample}(VarId, ObsId)$$

Algorithm 1: Stochastic Variational Inference for DFL PP

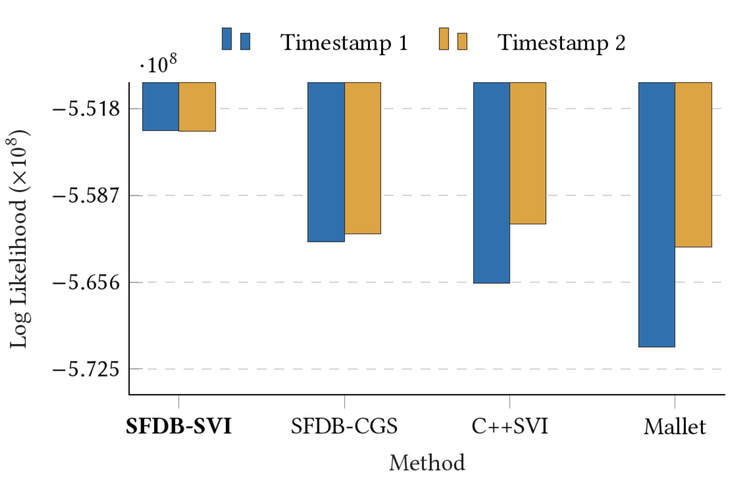
Input: A well-formed probabilistic program $(\mathcal{G}, \Phi)$

Randomly initialize $\mathbb{M}$;
repeat
  for $\Phi_{batch} \in \Phi$ **do**
    set $\hat{\mathbb{M}} = \mathbb{M}$
    repeat
      for $\phi_m \in \Phi_{batch}$ **do**
        Compute the local parameters $\boldsymbol{\lambda}'_m$
      for $\hat{\boldsymbol{\mu}}_n \in \hat{\mathbb{M}}$ **do**
        Compute the intermediate global parameters $\hat{\boldsymbol{\mu}}_n$
    until convergence of $\boldsymbol{\Lambda}'$ and $\hat{\mathbb{M}}$;
    for $\boldsymbol{\mu}_n \in \mathbb{M}$ **do**
      $\boldsymbol{\mu}_n := (1 - \rho_t)\boldsymbol{\mu}_n + \rho_t \hat{\boldsymbol{\mu}}_n$;
until convergence

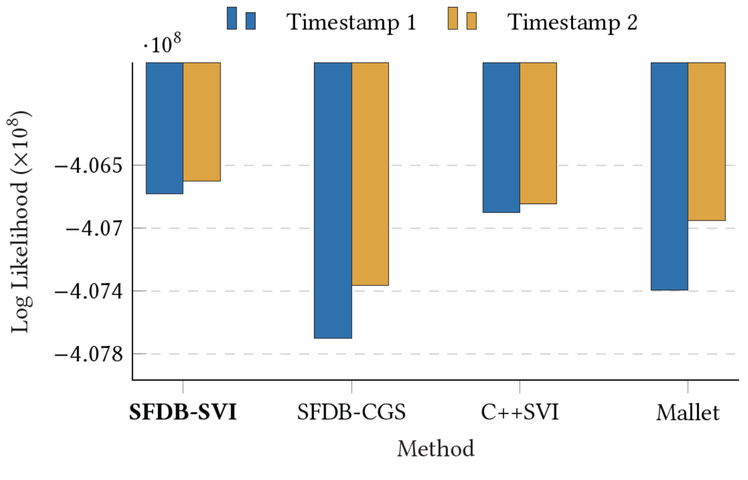
Algorithm-System Co-Design

Our prototype consists of a query-exec-engine that extends the one provided by Apache Arrow (Acero). We start with a probabilistic program and we use relational operators in ACERO to ground it into a large collection of propositional constraints.
All these constraints will share the same factorization as they were generated by the same structure. This means that we can use JIT-compilation[2] to build a function that is just perfect for sampling a solution for these constraints, that has this specific structure.
This allows us to compile this function once and execute it many many times, over all the constraints.

Results



(b) Wikipedia (200 topics)



(a) PubMed (100 topics)

Conclusion

We introduced a system that enables databases to perform statistical reasoning by developing a relational probabilistic programming framework built directly into a database query engine. We used stochastic variational inference for fast, scalable posterior approximation. We implemented this method and verified its correctness and scalability on a real-world, intractable inference task (LDA).

References

[1] Bárány, Vince, Balder Ten Cate, Benny Kimelfeld, Dan Olteanu, and Zografoula Vagena. "Declarative probabilistic programming with datalog." ACM Transactions on Database Systems (TODS) 42, no. 4 (2017): 1-35

[2] Finkel, Hal, David Poliakoff, Jean-Sylvain Camier, and David F. Richards. "Clangjit: Enhancing C++ with just-in-time compilation." In 2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC), pp. 82-95. IEEE, 2019.